01 / WHAT THE FILES MEAN

# Execution Evidence

What was tested, what the JSON records, and how to repeat it.

**Start here.** Execution evidence is a saved record of software running and its checks passing or failing. For BizWizard, the main backend operation log is `sandbox-operation-logs.json`. Benchmarks and simulation exports support it; they are not all the same kind of evidence.

## The six submitted files

These files are in `evidence/submission/`. Click a filename to open it in the repository; private-repository access is required.

| JSON file | What it records |
|---|---|
| `sandbox-operation-logs.json` | Migration and test commands, PASS/FAIL, timings, test summaries, expected versus observed numeric events, and test-database isolation. |
| `benchmark-results.json` | Offline comparison of fixed policies against baseline rules in three synthetic risk cases. No real AI call. |
| `benchmark-live-results.json` | The paid version of that comparison: model choices, results, fallback status, tokens and estimated USD cost. |
| `matched-evaluation.json` | A synthetic shop's saved inputs, rules-based choices, random seed, assumptions and paired simulation outcomes. |
| `matched-evaluation-live.json` | A saved shop-specific evaluation with live AI choices and eight paired outcome paths. |
| `matched-replay-verification.json` | Whether offline replay reproduced the saved live evaluation's outcome hash (result fingerprint). |

## How to read a report

**generatedAt** is its creation time. **source** identifies the code revision, source hashes and whether there were uncommitted changes. **report** contains the actual results; check its **success** field where present. Evaluation exports instead have a **status**, choices and outcome data.

For a numeric event, `expected` is the value asserted by the test; `observed` is what the backend returned. For example, `expected: 100, observed: 100, success: true` means that particular check matched. It does not mean every feature was tested.

**Reading comparisons:** lender results measure net credit return; borrower results measure borrowing cost; treasury results measure cash or collections; analyst results measure modeled returns. Positive differences favour the participant, including lower cost for the borrower. Do not add these different metrics together. A paired evaluation gives AI and rules the same simulated conditions, so their choices can be compared fairly within that model.

Recorded on 18 October 2026 (UTC), from clean revision `8f394d2d074b`. These are saved results, not a fresh test of the current release. Each JSON report includes the full revision and source hashes.

These are backend/sandbox records, not browser recordings, real bank transactions or proof of real-world profit. The checklist's execution-evidence item also names a demo video; JSON does not replace it. Its separate video item says optional, so confirm the organizer's interpretation if relying on that exemption.

02 / REPLAY AND BENCHMARK

# Try it without a database

Windows, Linux and macOS. No website or AI key needed for the first two runs.

## 1. Prepare the project once

Install Node.js 24 and Git. Clone the repository rather than downloading a ZIP; the reports record Git metadata. Run this in your terminal:

```
git clone https://github.com/mahfuz-saim/bizwizard.git
cd bizwizard/server
npm ci
npm run prisma:generate
```

If you already have the project and dependencies, just open a terminal in `server/`. Do not clone again. Private repositories require an invitation and Git authentication.

## 2. Reproduce the submitted live simulation

```
npm run evidence:finance -- --replay ../evidence/submission/matched-evaluation-live.json
```

**What happens:** the runner reads the saved shop snapshot, choices and random seed, recalculates the simulation, then compares its result hash with the saved outcome. It makes no new AI call and needs no database. Look for **Outcome hash match: PASS** and a new `matched-replay-verification.json`.

Replay checks the whole calculated outcome, not just its headline score. A changed outcome or invalid input can fail verification. A match proves repeatability, not authorship.

## 3. Run a fresh offline benchmark

```
npm run benchmark:finance
```

It compares four independent financial roles across three fixed risk cases. The output is labelled **NOT AI**. It saves `benchmark-results.json`; a successful run checks the comparison mechanics, not AI quality.

## Optional: use a real AI provider

Create `server/.env.benchmark` with your own key. This file is ignored by Git.

```
OPENAI_API_KEY=your-own-key
BENCHMARK_BUDGET_USD=0.10
```

```
npm run benchmark:finance -- --live
```

This makes paid calls using gpt-4o-mini. Invalid choices or fallback make a live run fail. Results can change; AI can lose. The budget and reported cost are application estimates, not a provider billing guarantee.

**Naming:** both benchmark modes write benchmark-results.json in new folders; the submitted live copy was renamed benchmark-live-results.json. **New shop evaluation:** in a synthetic/demo shop, open AI Agent Ledger > Run matched sandbox evaluation > download JSON. Live choices need backend AI access, budget, 60% sales-history coverage and 14 cash-history days. Run financing matching first for loan comparisons. Missing prerequisites can produce rules-based or pending results.

03 / DATABASE EXECUTION LOG

# Run the backend checks

Local PostgreSQL only. No AI key, VPS access or separately started backend needed.

## 1. Start PostgreSQL and create the test database

Install/start local PostgreSQL on port 5432. The README's Run 2 has Windows, Linux and macOS installation steps. Open its SQL console: Windows **SQL Shell (psql)**; Linux `sudo -u postgres psql`; macOS `psql -d postgres`. Set a password with `\password postgres` (Linux) or `\password` (macOS). Windows uses the installation password.

```
CREATE DATABASE simulator_test;
\q
```

If it already exists, continue; do not delete it. Complete page 2's dependency setup, then run one shell block below from `server/`. Use your **local** database login, never production credentials.

## 2. Windows: PowerShell

```
$login = Get-Credential -UserName postgres
$pw = [uri]::EscapeDataString($login.GetNetworkCredential().Password)
$env:EVIDENCE_DATABASE_URL = "postgresql://postgres:${pw}@127.0.0.1:5432/simulator_test"
npm run evidence:finance
Remove-Item Env:EVIDENCE_DATABASE_URL
Remove-Variable login, pw
```

## Or Linux/macOS: Bash

Run `bash` first if you use another shell. Use `postgres` on Linux; on macOS, use your local username (`whoami`). Password input is hidden.

```
read -r -p 'Database user: ' DB_USER
read -r -s -p 'Local password: ' DB_PASSWORD; printf '\n'
export DB_USER DB_PASSWORD
export EVIDENCE_DATABASE_URL="$(node -e '
const u = encodeURIComponent(process.env.DB_USER);
const p = encodeURIComponent(process.env.DB_PASSWORD);
console.log(`postgresql://${u}:${p}@127.0.0.1:5432/simulator_test`);
')"
npm run evidence:finance
unset EVIDENCE_DATABASE_URL DB_USER DB_PASSWORD
```

## 3. Check and keep the results

Expect **six PASS steps**: migrations, then finance-completion, finance-hardening, inventory-finance, simulator-workspace and matched-evaluation. The runner creates test data, exercises backend services and local HTTP routes, and saves `sandbox-operation-logs.json`; the matched test also saves `matched-evaluation.json`. Failed or skipped tests are not a successful run.

Each report has a new folder under `evidence/runs/`; follow the printed paths. Existing reports are not overwritten. The runner accepts only `127.0.0.1/simulator_test`, creates a fresh schema and retains it for inspection. Your login needs permission to create schemas. No app seed or production database is needed.

**If it fails:** check PostgreSQL is running, the password is correct and simulator_test exists. Keep the PASS/FAIL summary and any saved report; do not share connection strings. **For submission:** record final evidence from committed code, review the JSON and copy only selected reports into evidence/submission/. New timestamps, IDs and durations will differ. Runs on newer code are new evidence, not byte-for-byte copies of the saved October 18 reports.